

Adaptive Evidence Mesh for Near-Zero-Risk Connector Identification

Date: May 17, 2026

Author: Dr. T. Jerry Mahabub, Ph.D.

What the latest rev 2 run changes

The earlier diagnosis from the repo's own history was that the project's worst errors were driven less by "insufficiently fancy architectures" and more by **missing signal, weak supervision, tiny holdouts, and background bias**. The repo's classifier journey explicitly documents the old regime: only three short source videos, all from the same wood bench under one lighting condition, no SMA video at all, an 8-image phone-shot holdout, central pin or hole evidence that could be only about 5–10 pixels wide in the original crops, and many loose crops that let the model learn wood-grain and scene cues instead of connector structure. That document's conclusion was that the primary bottleneck was data, not model complexity. [1]

Based on the new class counts and confusion report from Kaggle (See Appendix A), that old conclusion has now been **partially overcome**. The class inventory is no longer starved in the way it was before, the 2.4 mm classes have much better support than in the earlier setup, SMA-M now exists as a real tenth class, and the confusion structure as reported is no longer showing catastrophic collapse. In other words, the enlarged dataset appears to have improved the model's **in-distribution organization of the class space**, especially for the 2.4 mm bottleneck we were watching most closely from the prior training runs showing this as a weakness due to lack of imagery, which the new image data downloaded earlier, mitigated this issue.

But that improvement does **not** mean the system is field-ready yet, and the hotel-TV Android failure is the proof. My own caveat is the right one: the Kaggle "confuser" score is optimistic because it includes training images. More importantly, the screenshot shown here for revision 2, shows exactly the kind of frame that will break a one-shot flat classifier: the connector face is small in-frame, blurred, at an oblique angle, attached to a cable assembly, and the center-contact evidence is not cleanly exposed. The app was highly confident in the **box** but only moderately confident in the **type**, and still returned a hard answer. That means the remaining bottleneck is no longer simply "too few examples of 2.4 mm." It is now a combination of:

- i. deployment-domain shift,
- ii. insufficiently evidence-aware decision policy, and
- iii. lack of targeted follow-up probes when the crop does not actually support a safe decision.

The repo's own design docs were written in anticipation of this problem by specifying:

- i. geometry verification,
- ii. confidence or ambiguity logic, and
- iii. valid outcomes such as
 - a. need_second_angle,
 - b. need_scale_reference,
 - c. ambiguous, and
 - d. unknown rather than forced guesses. [2]

Why the Android miss still happened

The current mobile path is temporarily simplified – for the purpose of testing singular classifier model inference to show how this is a dead-end going in this direction. Q.E.D.

The mobile export README says the on-device classifier was deliberately switched on **2026-05-17** from the legacy multi-head path to a **flat 10-class EfficientNetV2-S** model, with a YOLO11n single-class detector in front of it, all running in a Capacitor WebView via ONNX Runtime Web. That setup is excellent for getting a debuggable APK in our hands quickly, but it is not yet the architecture best matching the problem we are trying to resolve. [3]

A flat 10-class classifier is being asked to jointly solve several subtly different tasks at once: family, gender/contact, implicit size, and any latent thread or body cues that help distinguish near-neighbor precision connectors. The repo's taxonomy and classes metadata already decompose those concepts into separate attributes, but the current mobile path collapses them back into one softmax. That is acceptable for easy examples and debugging, but it is exactly the wrong decision rule for hard real-world connector photos where one part of the evidence is clear and another part is weak. A connector can be easy on **family** and hard on **gender**, or easy on **contact polarity** and hard on **size family**, or vice versa. A single flat answer hides that structure. [4]

The hotel-TV example is also a textbook case of **visibility mismatch**. In the screenshot, the connector is not a clean, fronto-parallel, high-resolution specimen image. It is a live camera capture in a hotel room, with blur, perspective, and real-world context. The repo's taxonomy already says that visually similar families may need scale, markings, manufacturer context, or another angle, and the training pipeline spec explicitly says the system should not force a guess when the evidence is insufficient. That means the bug here is not just "the classifier was wrong." It is that the inference stack did not yet have a mature **stop-or-refine policy**. [5]

What zero error really means in a mobile connector app

If the design goal is “keep adding evidence until the app can honestly say *nailed it*,” then the correct target is **not** “100% accuracy on every forced prediction from a single frame.” The correct target is **near-zero error on accepted predictions**, with the system allowed to keep refining, ask for more evidence, or abstain when the image does not support a safe decision. That is exactly the logic behind the repo’s confidence-state design and accuracy gates, and it matches the selective-classification literature, which improves reliability by letting a model reject uncertain examples rather than pretending certainty. [6]

This matters for everything that is listed, not just gender. If we want near-zero field error on:

- i. **connector type,**
- ii. **thread inference,**
- iii. **size family,**
- iv. **contact type, and**
- v. **related geometry-sensitive distinctions;**

Then the *app must be allowed* to say one of three things:

1. **Nailed it** — enough evidence is present and consistent.
2. **Need more evidence** — run more probes or ask for a better view and/or additional photos from the camera or via upload so the inference can work with very high accuracy.
3. **Cannot safely decide from this capture** — return `need_second_angle`, `need_scale_reference`, or `ambiguous`.

Without that three-way policy, “zero error” is not an engineering goal; it is just a wish. With that policy, it becomes a measurable system objective: minimize risk on accepted outputs while preserving as much coverage and usability as possible. That framing is also supported by calibration and selective-classification research, which shows that accuracy alone is not enough; confidence must correspond to actual risk. [7]

There is another important implication. The approach to “keep adding points on the grid or mesh” idea should be interpreted as **sequential evidence acquisition**, not a literal dense finite-element mesh over every pixel.

The right points are **semantic points and probes** tied to connector structure:

- i. center contact,
- ii. annular aperture,
- iii. thread ring,

- iv. hex-body flats,
- v. body-axis proportions, and
- vi. coupling cues.

Fine-grained vision research consistently shows that similar classes are best separated by focusing on localized, informative regions.

Adaptive-compute research shows that extra computation should be spent mostly on harder examples. [8]

The algorithm I recommend for rev 3

I recommend replacing the current “detector + one flat classifier = final answer” logic with an **Adaptive Semantic Evidence Mesh (ASEM)**.

The idea is to keep the current shipping path as the fast front end, but make it stage zero of a multi-stage decision process. Easy captures still exit early. Hard captures accumulate progressively more evidence until the system can either declare **nailed it** or responsibly request more information. This is the closest rigorous implementation of the architecture being proposed / described.

It also fits the repo’s target plan of:

- i. detector,
- ii. attribute classifier,
- iii. geometry or measurement module,
- iv. spec constraints, and
- v. confidence/abstention gate. [9]

The stages should work in sequential / hierarchical order like this:

Stage one is the existing YOLO11n detector plus flat 10-class EfficientNetV2-S classifier. It produces a box, a top-k class posterior, and a calibrated uncertainty estimate. If the class is already high-confidence and the margin over the next alternative is large, the system can stop early.

Stage two is a lightweight attribute refiner. Instead of predicting only a compound class, it predicts a small set of reusable attributes such as family, gender/contact, and view sufficiency. This stage should be cheap and should run only when stage one is not decisive. The good news is that the repo already has the scaffolding for this: the multi-head model, the cloud trainer, the head vocabularies, and masked missing-label handling are already in place. [10]

Stage three is the actual “point mesh” layer: a small library of semantic probes or landmarks that are chosen adaptively depending on the current confusion set. For the current ten-class problem, the minimum viable probe set is:

- a **center-contact probe** for pin versus socket versus occluded center,
- an **aperture / dielectric annulus probe**,
- a **thread-ring outer-diameter probe**,
- a **thread-pitch-density proxy** rather than exact metrology from a raw phone image,
- a **hex/body-flat span probe**,
- and, when the angle permits, a **body-axis / shoulder / visible-length probe**.

These are the “points” that matter. They form a sparse semantic mesh over the connector face and body. The repo’s geometry schema already has places for thread diameter, pitch or count, body length, hex size, aperture, and a flag saying a calibrated reference is required. [11]

Stage four is a spec and geometry gate. The repo’s taxonomy and `connectors.yaml` already encode compatibility, polarity rules, and family-specific notes. That means once the top candidates are narrowed to a small set, we can rule out impossible or evidence-inconsistent hypotheses. For example, if the refiner thinks the center looks like a socket but the current top class hypothesis requires a pin, the posterior for that class should crater. If a candidate distinction is scale-sensitive and no scale reference is present, confidence should stop rising and the system should route to `need_scale_reference` rather than hallucinating precision. [12]

Stage five is the “nailed it” gate. This should be a selective-classification step, not just a softmax threshold. The system should only emit “nailed it” when the accumulated evidence passes all three tests:

- calibrated confidence is high,
- the margin to the second-best consistent hypothesis is high,
- and the specific required evidence for the requested claim is actually visible.

That last condition is critical. The app must not claim a contact gender if the center-contact visibility probe says the center is blurred, occluded, or too small to resolve.

A practical decision rule looks like this:

```
detect -> classify_flat -> calibrate

if conf >=  $\tau_{\text{accept}}$  and margin >=  $\tau_{\text{margin}}$  and visibility_ok(top1):
    return NAILED_IT(top1)

K = top_k_consistent_classes

run attribute_refiner

while budget remains:
    j = argmax expected_information_gain(probe_j | K, visible_regions) / cost_j    z_j =
```

```

run_probe_j(crop)
    update posterior over K      if conf >=  $\tau_{\text{accept}}$  and margin >=  $\tau_{\text{margin}}$  and
required_evidence_visible(top1):
    return NAILED_IT(top1)

if scale_needed(K) and no_scale_reference:
    return NEED_SCALE_REFERENCE

if angle_needed(K) or center_visibility_low:
    return NEED_SECOND_ANGLE

return AMBIGUOUS(top_alternatives)

```

This is not just a heuristic. It is the connector-specific version of adaptive feature acquisition and early-exit inference: gather more evidence only when the current evidence cannot support the desired risk level. [13]

For a more formal math-based explanation and derivation of how the ASEM process() works, please refer to [Appendix B](#).

How to fit this into the current repo without breaking the debug path

The repo is unusually well-positioned for this upgrade because its scaffolding already points in the right direction. The first rule should be: **do not throw away the current flat-classifier APK path**. Keep it as stage one, preserve /predict compatibility, and add richer structured output alongside the old fields exactly as the implementation plan requires. That preserves the working demo while turning it into the front end of a safer decision stack. [14]

The fastest next engineering step is to restore **attribute-wise reasoning** without requiring a full relabeling campaign. Because `classes.yaml` already encodes family and gender metadata for the current ten classes, we can derive supervision for a **family head** and **gender/contact head** immediately from the current flat labels. The multi-head label system also supports missing targets, which means we can add new heads later, such as visibility, orientation, or mount-style, before the entire dataset is comprehensively relabeled. [15]

The next step after that is to add a **small probe model library** to the mobile bundle rather than replacing the whole backbone again. The current mobile package already carries a roughly 10 MB detector and roughly 77 MB classifier. Adding one or two much smaller quantized ONNX probe models for center-contact and annular thread/diameter inspection is operationally easier than betting everything on yet another single monolithic classifier. [16]

On the data side, use the repo's own instance-manifest and specimen-aware dataset builder as the canonical pipeline. The builder emits `attributes.csv`, performs deterministic specimen-aware splitting, and supports single-class detector mode and family-aware classification mode. That means we can attach a new sparse annotation file for semantic probes—landmark points, probe labels, or visibility flags—without disrupting

the core detector/classifier dataset. Because the builder already guards against specimen leakage, it is the right place to attach any new evidence-annotation program. [17]

I would add one more training track to the repo: a **hard-case mining loop**. Every field capture where the flat classifier is below the “nailed it” threshold should be saved into a curated hard-case pool, grouped by specimen and capture session. Those cases should then be tagged for the specific failure mode: blur, poor center visibility, scale ambiguity, side-angle needed, family confusion, or gender confusion. That hard-case pool should become the primary dataset for the probe models and the confidence gate. This is much more aligned with the real objective than simply growing the global dataset indiscriminately.

Validation gates and the path to a true “nailed it” state

The next milestone should not be “APK compiles and classifier score is high on Kaggle.” It should be “accepted predictions are trustworthy in field conditions.” The repo already lists the right ingredients for honest evaluation: detection mAP, per-head accuracy, macro F1, top-k accuracy, confusion matrices, calibration error, abstention-aware correctness, latency, model size, and mobile constraints. I would keep all of those and add two more metrics that matter specifically for the design spec:

- **coverage at nailed-it risk:** of all captures, how many end in a “nailed it” decision below a target field error rate,
- **average probe cost to acceptance:** how many additional probes, zooms, or retakes were needed before the system safely accepted. [18]

We should also split the evaluation into three tiers:

- **in-distribution** curated data,
- **real phone captures** with realistic backgrounds,
- **deployment stress cases** like hotel TVs, routers, cable backs, poor lighting, soft focus, oblique angles, and partially attached cables.

The field-ready gate should be driven by the second and third tiers, not by the first. The repo’s accuracy gates already state that large, statistically meaningful real-phone holdouts are required before making strong production claims. [19]

One more point deserves to be said plainly: exact thread height, pitch, and spacing from arbitrary handheld phone photos are not always physically recoverable from one frame, especially without scale and with motion blur. The right design response is **not** to force a false sense of exactness. It is to use robust visual proxies when possible, absolute metrology only when a reference is available, and `need_scale_reference` or `need_second_angle` when the image cannot support a stronger claim. The training pipeline spec explicitly warns against claiming exact physical dimensions without scale, and the OpenCV ArUco documentation confirms that fiducials and boards are the right route when precise pose or scale reasoning is needed. [20]

The most important conclusion is this: The new data run shows that the old “data, not bigger models” diagnosis was correct **for the previous bottleneck**, and the 2.4 mm confusion improvements appear to validate that. But the Android field miss shows that the next bottleneck is now different. It is no longer primarily class scarcity. It is the absence of an **evidence-aware inference policy** that knows when to stop, when to zoom in, which semantic points to inspect next, and when to refuse to guess. The correct architectural upgrade for this repo is therefore an **adaptive semantic evidence mesh**: start macroscopic, progressively zoom into connector-critical regions, accumulate targeted evidence, and declare **nailed it** only when the evidence supports the claim at the risk level we want. [21]

Appendix A: Kaggle Confusion Cell Output for rev 2

scored 50/635

scored 100/635

scored 150/635

scored 200/635

scored 250/635

scored 300/635

scored 350/635

scored 400/635

scored 450/635

scored 500/635

scored 550/635

scored 600/635

9-Class Evaluation (*Actually 10-class*)

- Overall accuracy: *0.9732*

Per-class

Class	Support	Precision	Recall
1.85mm-M	86	0.977	0.988
1.85mm-F	82	1.000	0.915
2.4mm-M	62	0.983	0.935
2.4mm-F	59	0.951	0.983
2.92mm-M	54	0.945	0.963
2.92mm-F	66	0.957	1.000
3.5mm-M	70	0.958	0.986
3.5mm-F	62	1.000	0.984
SMA-F	54	0.964	1.000
SMA-M	40	1.000	1.000

Notable confusions (adjacent size / M-vs-F)

- 1.85mm-F -> 1.85mm-M: 2

- 1.85mm-F -> 2.4mm-F: 2

- 1.85mm-F -> SMA-F: 2
- 2.4mm-M -> 2.92mm-M: 2
- 2.92mm-M -> 3.5mm-M: 2
- 1.85mm-M -> 2.4mm-M: 1
- 1.85mm-F -> 2.92mm-F: 1
- 2.4mm-M -> 2.4mm-F: 1
- 2.4mm-M -> 3.5mm-M: 1
- 2.4mm-F -> 2.92mm-F: 1
- 3.5mm-M -> 2.92mm-M: 1
- 3.5mm-F -> 2.92mm-F: 1

NOTE: scored over ALL images including training data -> headline accuracy optimistic; read the CONFUSION STRUCTURE, not the number.

This is the payoff — and it's a strong result. (Ignore the "# 9-Class" header; that's a hardcoded title string in the report module — the table is correctly 10 classes. I'll fix that cosmetic in rev 3.)

The 2.4mm bottleneck is resolved:

Class, then (9-cl, n), now (10-cl, n)

2.4mm-F, R 0.80 (n=10), R 0.983 (n=59)

2.4mm-M, R 0.938 (n=16), R 0.983 (n=62)

2.4mm-F went from the worst class to essentially solved — exactly what the 10→59 data bump was supposed to do.

SMA-M (the new 10th class) is perfect — P/R 1.000 at only 40 images, and there is zero SMA-M ↔ SMA-F confusion (not in the notable list). Adding the 10th class cost nothing.

Gender is clean: the only same-size M/F error is 1.85mm-F → 1.85mm-M ×2. Everything else is size-adjacent, not gender.

New weakest class: 1.85mm-F (recall 0.915) — it's now the soft spot, bleeding a little into 1.85mm-M, 2.4mm-F, SMA-F,

2.92mm-F (~7 of the ~17 total errors). Likely from the new batch's different lighting/backgrounds introduced harder 1.85mm-F examples. It's the next data/collection target, but it's a minor effect — every class is now recall ≥ 0.915.

Important caveat: 0.9732 is over-all 635 images including training data → optimistic.

Bottom line:

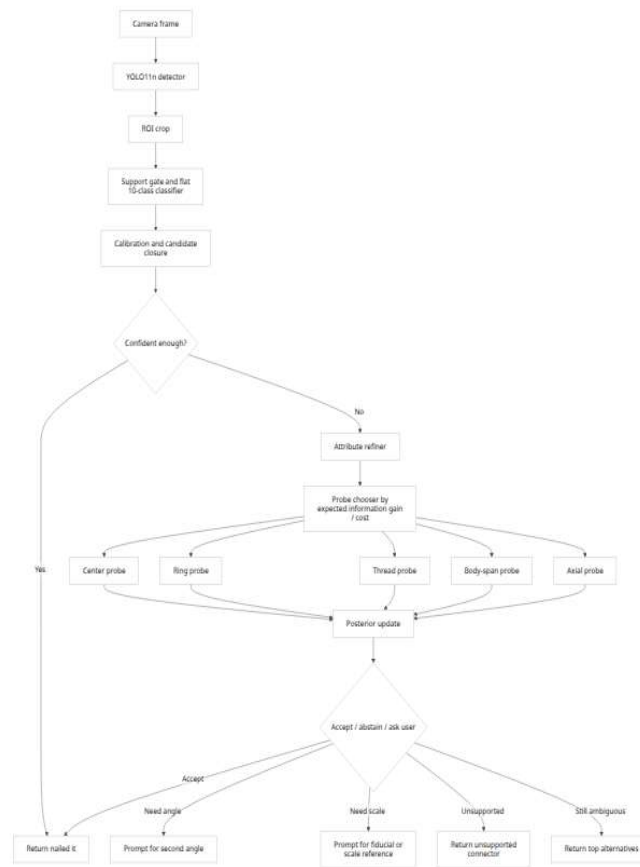
The confusion structure above is reliable regardless, and it's good: well-distributed, no class collapse, bottleneck closed. Structurally this is good, systematically and single model functionality-wise – it fails.

Next action: We need to collect more real photos of 1.85mm-F and the 2.4/2.92/3.5 classes to push past current baseline, which may work well on real connectors - building the debug.apk now for android testing...

Appendix B: Adaptive Semantic Evidence Mesh design

The right interpretation of the “keep adding points to the grid or mesh until it says nailed it” idea is **not** a literal finite-element mesh over the image. The literature closest to my idea is the literature on **fine-grained localized evidence** and **adaptive computation**. RA-CNN and NTS-Net both improve fine-grained recognition by recursively or explicitly focusing on the small number of localized regions that separate similar classes, rather than treating all pixels as equally useful. BranchyNet and MSDNet, meanwhile, formalize the idea that easy examples should exit early and difficult examples should consume additional compute. Sequential cost-sensitive feature acquisition and adaptive classification under a budget go one step further: acquire expensive evidence only when it is expected to reduce uncertainty enough to justify its cost. That is the right mathematical framing for the system. [18]

The proposed architecture is an **Adaptive Semantic Evidence Mesh**, or ASEM. It is adaptive because it only spends extra compute on uncertain cases. It is semantic because its “points” are not arbitrary pixels; they are connector-relevant landmarks and regions such as center contact, annular aperture, thread ring, body span, and axial shoulders. It is a mesh because the evidence graph grows only as needed, from a coarse global hypothesis toward finer structural evidence.



The most important design choice is to add an explicit **support gate** before or alongside the current flat softmax. The current 10-class mobile classifier has no unknown or unsupported class in `classes.yaml`, while the broader taxonomy and implementation plan explicitly require unsupported and ambiguous outcomes. ASEM therefore adds a binary or small multiclass gate that estimates whether the crop belongs to the current ten-class support set at all. That directly addresses cases like the hotel-TV example, which appears plausibly out-of-scope for the current ten supported classes. [19]

The **stage definitions** should be concrete and minimal:

Stage	Purpose	Runs when	Output
Detector	Localize connector-like ROI	Always	bbox, box confidence
Support gate	Decide supported vs unsupported vs uncertain	Always	$p(\text{in_support})$, $p(\text{unsupported})$
Flat classifier	Cheap global hypothesis over 10 current classes	Always	calibrated class posterior
Candidate closure	Reinsert structurally nearby hypotheses	If not accepted	candidate set K

Attribute refiner	Family, gender/contact, view sufficiency, maybe support	If uncertainty remains	head posteriors
Probe runner	Center, ring, thread, body-span, axial	Only as needed	semantic evidence z_j
Spec gate	Enforce consistency with known connector constraints	After each update	compatibility mask
Decision gate	Accept, ask for more view, ask for scale, unsupported, ambiguous	After each update	final app state

The **candidate closure** step deserves special emphasis. A flat classifier's top-k list may omit the very class that a later probe would vindicate. For example, if the flat softmax says 2.4mm-M but a center probe later sees a socket, the correct answer might be 2.4mm-F even if that class was not initially top-3. Therefore, ASEM should not restrict later reasoning to the current top-k only. It should expand the candidate set by adding the **same-family opposite-gender neighbor**, the **adjacent precision-size neighbors**, and the **unsupported/OOD sentinel**. For the current ten-class support set, a good closure rule is $k = 3$ to 5 plus structural neighbors. This prevents the system from getting trapped by an early wrong gender or adjacent-size hypothesis.

The **posterior update rule** is Bayesian in spirit even if the individual likelihood terms are learned discriminative modules. Let C be the connector class, $\pi_t(c)$ the current posterior after evidence set D_t , and z_j the new output from probe j . Then the update is:

$$\pi_{t+1}(c) = \frac{\pi_t(c) L_j(z_j | c)^{w_j} A(c | \text{attribute heads}) S(c | \text{spec constraints})}{\sum_{c' \in K_t} \pi_t(c') L_j(z_j | c')^{w_j} A(c' | \text{attribute heads}) S(c' | \text{spec constraints})}$$

Here, L_j is the probe likelihood or likelihood ratio, w_j is a learned or tuned probe weight, A is the attribute-consistency factor, and S is a hard or soft spec-compatibility factor derived from the taxonomy and connector specs. The full derivation appears in Appendix B.

The **probe selection policy** should maximize **expected information gain per cost**:

This is a **cost-normalized information-gain acquisition rule**.

$$j^* = \arg \max_{j \in \mathcal{J}_{\text{available}}} \frac{\mathbb{E}_{z_j} [H(\pi_t) - H(\pi_{t+1} | z_j)]}{\text{cost}(j)}$$

Plain-English meaning

Choose the next available diagnostic action, observation, view, model head, sensor query, or connector-inspection point j that gives the **largest expected reduction in uncertainty per unit cost**.

In the SMA connector detector context:

“Look next at the feature/point/view/model-query that is expected to reduce classification uncertainty the most, after accounting for how expensive it is to compute.”

Term-by-term breakdown

Term	Meaning
j^*	Best next action/query/inspection point
$\mathcal{J}_{\text{available}}$	Set of currently available candidate actions
π_t	Current class belief distribution over connector classes at time t
$H(\pi_t)$	Current entropy/uncertainty of the model’s belief
z_j	Possible observation obtained by taking action j
π_{t+1}	z_j
$H(\pi_{t+1})$	z_j
$E_{z_j}[\dots]$	Average expected uncertainty reduction over possible outcomes
$\text{cost}(j)$	Compute, latency, battery, memory, or sensor cost of action j

where H is entropy and $\text{cost}(j)$ includes not only compute time but also user-friction cost if the probe requires a recapture, zoom, or scale card.

That policy can be implemented approximately rather than exactly. On mobile, the practical approximation is to use a **pairwise confusion policy**. If the current confusion is mainly same-size M vs F , run the **center probe** first. If the confusion is mainly 2.4 vs 2.92 vs 3.5 , run the **ring** and **body-span** probes first. If the candidate set is drifting toward unsupported or the visible geometry is inconsistent with the current support set, elevate the **support gate** or return `unsupported_connector`. This approximates expected information gain while remaining simple enough for the current repo and APK.

The repo’s own taxonomy strongly supports that pairwise strategy. `connectors.yaml` explicitly notes that 2.92 mm can be visually similar to SMA and 3.5 mm and may require scale or markings, and that 1.85 mm likely needs scale or markings. So ASEM should not try to force exact size-family claims when the visible evidence is fundamentally insufficient. [20]

The **decision rule** should use calibrated confidence, not raw softmax alone. Guo et al. show that modern neural networks are often poorly calibrated and that temperature scaling is a strong practical baseline. Selective-classification work then provides the operational framing: tune confidence thresholds not to maximize raw accuracy, but to achieve acceptable **risk at a chosen coverage level**. In this project, that means “nailed it” should be a **selective acceptance state**, not just “top-1 is largest.” [21]

A practical threshold policy for the current repo is:

Quantity	Recommended range	Meaning
k for candidate closure	3–5	top candidates before structural neighbors
$\tau_{\text{accept_family}}$	0.99–0.995	accept family-only claim

$\tau_{\text{accept_full}}$	0.995–0.999	accept full type + gender claim
τ_{margin}	0.20–0.35	minimum posterior gap to runner-up
$\tau_{\text{support_unsupported}}$	0.60 early, 0.80 after one probe	return unsupported_connector
$\tau_{\text{visibility}}$	0.85–0.90	minimum visibility to claim center-based gender
automatic probe budget	2–3 probes	per frame before asking user
total evidence budget	4–6 points	across frame plus guided recapture

These are **recommended tuning ranges**, not fixed truths. They should be tuned on real-phone validation data against the repo’s selective-accuracy goals.

The hotel-TV failure illustrates the rule nicely. The current app stopped at a flat top-1 of **68.8%**, which is far below any credible nailed it threshold. Under ASEM, a case like that would almost certainly route to **additional probes** first. If the support gate rose strongly, it would return unsupported_connector. If center visibility or size evidence remained poor, it would route to need_second_angle or need_scale_reference rather than declaring 2.4mm-M.

Probe models, rollout, and validation

The minimal evidence library needed is feasible on-device if the probes are tiny and conditional. The five requested probes—center, ring, thread, body-span, axial—map naturally onto the repo’s schema and can be trained and exported without discarding the current flat APK path. I recommend those five plus one additional tiny **support gate**, because unsupported handling is critical if we want connector-type error near zero.

Probe	Primary question	Crop definition	Suggested output	Target model size	Added latency target	Typical best use
Support gate	Is this even in the current 10-class support set?	full ROI	in-support / unsupported / uncertain	0.5–1.0 MB INT8	2–4 ms	OOD rejection
Center probe	Pin, socket, reversed contact, or unresolved?	tight central patch	contact logits + visibility	0.4–0.8 MB INT8	2–4 ms	M vs F
Ring probe	What are the inner/outer annulus and aperture cues?	face-centered patch	radii, ratios, confidence	0.8–1.2 MB INT8	4–6 ms	size-family refinement
Thread probe	Are visible threads present and what coarse density regime?	ring-side patch	none / coarse / fine / hidden	0.6–1.0 MB INT8	3–5 ms	type/thread cue

Body-span probe	What is the body or hex span relative to face?	wider ROI crop	normalized span, confidence	0.6–1.0 MB INT8	3–5 ms	size-family refinement
Axial probe	Is shoulder/body-length evidence present?	side-biased or elongated crop	visible-length cues	0.8–1.2 MB INT8	4–7 ms	oblique-view disambiguation

These size and latency numbers are proposed engineering **targets** for a mid-range Android device, not measurements from the current repo. The design goal should be to keep the incremental probe bundle below roughly **6 MB** total and keep the worst-case automatic multi-probe overhead within a tolerable budget for a WebView-based ONNX Runtime deployment. The mobile README shows the current detector-plus-flat-classifier asset load is already around **87 MB**, so the probe library must stay small enough not to turn the APK into an unmaintainable artifact. [22]

The probe training strategy should be incremental and data efficient. Because the dataset builder already emits `attributes.csv` and follows specimen-aware splitting, we can attach new sparse probe annotations to the same manifest pipeline rather than inventing a second training universe. The minimal annotation plan is:

- for **center**: a point, tiny mask, or categorical tag for pin, socket, reversed, hidden, plus `center_visibility`;
- for **ring**: inner and outer circle or ellipse parameters, or weak labels yielding normalized aperture and ring ratios;
- for **thread**: coarse thread visibility and density bins rather than exact pitch;
- for **body-span**: one- or two-line endpoints for apparent face/body span or outer-body hex flats;
- for **axial**: endpoints or categorical visibility tags for shoulder and visible body-length cues.

The strongest data source for these labels is not the whole dataset. It is the **hard-case pool**: low-margin images, mobile screenshots, out-of-distribution cases, and confusion-pair examples. That is where each probe will learn the highest-value evidence fastest.

The repo history strongly supports prioritizing **real phone photos** over more architectural churn. The classifier journey explicitly says that family was partly learnable from the old videos, but gender required photo-resolution training data, and that data diversity across scenes and lighting is what raises the generalization ceiling. The implementation plan also states that synthetic data can help but does not replace a real phone-photo holdout. [23]

The rollout plan should therefore be truly incremental:



The concrete implementation sequence I recommend is:

Rollout step	Repo touchpoints	Why first
Add confidence gate and abstention states to current app	exports/mobile/, /predict response schema	fastest risk reduction
Add support gate	mobile decode + flat classifier wrapper	prevents forced OOD guesses
Restore multi-head refiner for family, gender, visibility	label_encoding.py, model_multihead.py, train_multihead.py	uses existing scaffolding
Add center + ring probes	new micro-models + geometry fields	highest immediate value
Add thread/body-span/axial probes	new probe package + app orchestration	resolves adjacent precision classes
Add guided capture prompts	result card and camera UX	makes abstention actionable
Add optional scale marker workflow	geometry + OpenCV fiducials	only when absolute dimension needed

The OpenCV ArUco documentation supports the scale-reference path. OpenCV’s ArUco FAQ notes that single markers can be used to label objects, while the detection and calibration tutorials describe using markers and ChArUco boards for pose estimation and calibration. That is exactly the right mechanism when we genuinely need calibrated measurements rather than scale-free visual proxies. [24]

Validation must move from notebook-style optimism to risk-aware field evaluation. The implementation plan already tracks many of the right metrics, but ASEM needs a few more that are specific to sequential evidence systems.

Metric	What it answers	Keep or add
mAP, precision, recall for detector	are ROIs found reliably?	keep
per-class precision/recall/F1	where do label confusions remain?	keep
per-head accuracy	are attribute heads trustworthy?	keep
top-k accuracy	is the candidate set healthy?	keep
ECE	are confidences calibrated?	keep
abstention-aware accuracy	when the model answers, how often is it right?	keep
coverage at target risk	how often can it say “nailed it” safely?	add

probe-cost to acceptance	how much extra evidence does hard inference need?	add
AURC or related RC metric	how well does confidence trade off with coverage?	add
OOD rejection accuracy	how often unsupported connectors are properly rejected	add

The calibration and selective-classification literature give's the right conceptual backbone here. Calibration work recommends post-hoc temperature scaling as a strong baseline for turning logits into more reliable probabilities, while selective-classification work shows why risk-coverage tradeoffs should be measured rather than hidden. Recent selective-classification evaluation work also emphasizes risk-coverage metrics such as AURC variants as meaningful summaries of abstention behavior. [25]

The test tiers should be explicitly separated:

Tier	Content	Promotion role
In-distribution curated	clean class-balanced images and known-support connectors	debug and ablation only
Real-phone holdout	phones, varied surfaces, varied lighting, multiple angles	main promotion gate
Stress and OOD	blur, oblique, hotel TVs, consumer coax, partial occlusion, scale-missing, unsupported connectors	safety gate

The repo's own implementation plan is already aligned with this hierarchy. It explicitly says that 99.99% is aspirational, that true claims require large, diverse, independently held-out validation, and that synthetic data does not replace a real phone-photo holdout. [26]

Two limitations should be stated plainly.

First, the uploaded notebook is **not fully reproducible** as an audit artifact because it clones repo HEAD rather than pinning a commit and because some cell sources are stored with literal . . . elisions. That means I can verify the stored outputs and visible commands, but not every omitted intermediate line.

Second, exact thread pitch, exact thread height, and exact size-family determination are not always recoverable from a single handheld blurred phone frame without scale. The right design response is not to pretend otherwise. It is to use coarse thread evidence when visible, absolute metrology only when a fiducial or scale is present, and need_second_angle or need_scale_reference when the evidence cannot support a stronger claim.

Appendix B1 - mapping from equations to repo files

The table below maps the proposed ASEM mathematics to the repo's present code and schema anchors. Where a mapping is not yet implemented, I mark it as a recommended insertion point rather than a current feature.

ASEM concept	Repo anchor	Current status
Support set S of current flat classes	training/configs/classes.yaml, training/rfconnectorai/data/classes.py	implemented for 10 flat classes [10]
Flat prior $p_{\text{flat}}(c x)$	training/rfconnectorai/classifier/train.py, current mobile classifier export	implemented [27]
Attribute priors $p_h(a_h x)$	label_encoding.py, model_multihead.py, train_multihead.py	scaffolded, not current mobile path [28]
Missing-label masking	label_encoding.py, train_multihead.py	implemented [29]
Specimen-aware split discipline	build_yolo_dataset.py	implemented [12]
Geometry evidence storage	schemas/instance.py GeometryLabel	implemented schema, not yet used by probe stack [30]
Spec compatibility factor $S(c)$	specs/connectors.yaml	implemented knowledge base, not yet full inference gate [15]
Confidence state and abstention	connectors.yaml, IMPLEMENTATION_PLAN.md	planned and partially documented; needs stronger mobile enforcement [31]
ONNX export path for added probes	export/export_mobile.py, exports/mobile/README.md	export scaffolding exists; real cloud export required [32]
/predict compatibility	predict_service.py, IMPLEMENTATION_PLAN.md	hard constraint already documented [33]
Evaluation metrics for selective operation	IMPLEMENTATION_PLAN.md reports and gates	mostly defined, but coverage@risk and probe-cost should be added [34]

Appendix B2 - formal mathematical definition of ASEM

Notation glossary

Symbol	Meaning
x	detector ROI crop from the camera frame
S	current supported flat class set, initially the 10 classes in <code>classes.yaml</code>
$\perp$	unsupported or out-of-distribution sentinel class
C	random variable over $S \cup \{\perp\}$
D_t	accumulated evidence after t stages
$\pi_t(c)$	posterior probability $P(C=c \mid D_t)$
$l_{\text{flat}}(c)$	uncalibrated flat-classifier logit for class c
T	temperature-scaling parameter
K_t	active candidate set at step t
h	attribute head such as family or side-a gender
$a_h(c)$	attribute value implied by class c for head h
$p_h(v x)$	attribute-head posterior for value v
j	probe index from the probe set
z_j	observation emitted by probe j
$L_j(z_j c)$	likelihood contribution of probe observation z_j under class c
$\text{cost}(j)$	probe cost including latency and optionally user friction
$H(\pi)$	entropy of posterior π
τ_{accept}	acceptance threshold
τ_{margin}	posterior-margin threshold
τ_{vis}	minimum visibility or sufficiency threshold
A, B	SPRT-like upper and lower odds bounds
ϵ_{probe}	minimum acceptable information-gain-per-cost score

State space and initial posterior

ASEM begins with a detector crop x , a support-gate score $q_{\text{support}} = P(C \in S \mid x)$, and flat logits $l_{\text{flat}}(c)$ for $c \in S$. Temperature scaling gives the calibrated supported-class prior

$$p_{\text{flat}}(c \mid x) = \frac{\exp(l_{\text{flat}}(c)/T)}{\sum_{c' \in S} \exp(l_{\text{flat}}(c')/T)}.$$

Then the full initial posterior over $S \cup \{\perp\}$ is

$$\pi_0(c) = q_{\text{support}} p_{\text{flat}}(c \mid x), \quad c \in S$$

and

$$\pi_0(\perp) = 1 - q_{\text{support}}.$$

If a dedicated support-gate is not yet available, $\pi_{\theta}(\perp)$ can be approximated by a calibrated energy or distance score in the flat embedding space, but a trained support-gate is preferred.

Candidate closure

Let $\text{TopK}(\pi_{\theta}, k)$ be the top-k supported classes by posterior. The candidate closure operator is

$$K_0 = \text{Close}(\text{TopK}(\pi_0, k)) \cup \{\perp\}.$$

Close adds structural neighbors: same family opposite gender, adjacent precision-size neighbors, and other connector-spec neighbors that are plausible under the taxonomy. This avoids irreversible early pruning.

Attribute fusion

Suppose a set of attribute heads $\mathcal{H}$ outputs posteriors $p_h(v|x)$. For each class c , define the attribute-consistency factor

$$A(c) = \prod_{h \in \mathcal{H}} p_h(a_h(c) | x)^{\lambda_h},$$

where $\lambda_h \geq 0$ is a head weight. If a particular attribute is not applicable or not visible, its contribution can be skipped or replaced by a neutral factor of 1.

The posterior after attribute fusion is

$$\tilde{\pi}_0(c) = \frac{\pi_0(c) A(c)}{\sum_{c' \in K_0} \pi_0(c') A(c')}.$$

Probe update

Each probe j emits observation z_j . Examples:

- center probe: logits over pin, socket, reversed, hidden, plus center visibility;
- ring probe: normalized aperture and annulus radii;
- thread probe: visibility and coarse density bin;
- body-span probe: normalized span estimate;
- axial probe: visible-length and shoulder cues.

Define the learned likelihood factor $L_j(z_j | c)$. The Bayesian-style update is

$$\pi_{t+1}(c) = \frac{\pi_t(c) L_j(z_j | c)^{w_j} S_j(c, z_j)}{\sum_{c' \in K_t} \pi_t(c') L_j(z_j | c')^{w_j} S_j(c', z_j)},$$

where w_j is a probe weight and $S_j(c, z_j)$ is a hard or soft compatibility factor. For example, if the center probe confidently predicts socket, then any class whose required contact state is incompatible with a visible socket should receive a severe penalty.

In implementation, L_j does not need to be a generative density in the classical sense. It can be a calibrated discriminative likelihood ratio, a learned compatibility score exponentiated into positive space, or a tabulated score based on probe regression residuals.

Expected information gain and probe choice

From earlier, the ideal probe selector is

$$j^* = \arg \max_{j \in \mathcal{J}_{\text{available}}} \frac{\mathbb{E}_{z_j}[H(\pi_t) - H(\pi_{t+1} | z_j)]}{\text{cost}(j)}$$

This is the expected decrease in posterior entropy per unit cost.

A practical approximation is to use the current candidate set and a probe-utility table derived from validation data. For example:

- if dominant confusion is same-size M/F, assign highest utility to center probe;
- if dominant confusion is 2.4 vs 2.92 vs 3.5, assign highest utility to ring and body-span;
- if support posterior for $\perp$ is already large, assign highest utility to support confirmation or return unsupported directly.

This is a **cheaper, class-focused approximation** of the previous entropy-based acquisition rule.

$$j^* = \arg \max_j \frac{\sum_{c \in K_t} \pi_t(c) u_j(c, K_t)}{\text{cost}(j)}$$

Meaning

Choose the next action/query/inspection j that has the highest **expected discriminative utility per unit cost** over the current candidate class set K_t .

In SMA connector detection terms:

“Among the connector classes still plausible right now, choose the next inspection that is most useful for separating those candidates, weighted by how likely each class currently is.”

where $u_j(c, K_t)$ is learned from validation as the average risk reduction when probe j is applied under confusion context K_t .

Stopping criteria

The **second-best class** from the current belief distribution π_t .

A clean version is:

$$c^* = \arg \max_{c \in K_t} \pi_t(c), c^{(2)} = \arg \max_{c \in K_t \setminus \{c^*\}} \pi_t(c)$$

Meaning

Symbol	Meaning
c^*	Most likely class under the current belief distribution
$c^{(2)}$	Second-most likely class
$\pi_t(c)$	Current probability assigned to class c
K_t	Current active candidate class set

So:

c^* = current best class prediction
 $c^{(2)}$ = strongest competing class

SMA detector interpretation

For the SMA connector classifier:

c^* = model's current best connector guess
 $c^{(2)}$ = model's closest competing connector class

Example:

$\pi_t(\text{SMA male straight}) = 0.46$
 $\pi_t(\text{RP-SMA male straight}) = 0.39$
 $\pi_t(\text{SMA female straight}) = 0.09$
 $\pi_t(\text{SMA right-angle}) = 0.06$

Then:

$c^* = \text{SMA male straight}$
 $c^{(2)} = \text{RP-SMA male straight}$

Why $c^{(2)}$ matters

The pair:

$(c^*, c^{(2)})$

defines the **primary confusion pair**.

That is exactly where the next inspection should focus. Instead of asking, “What is the best next feature for all 10 classes?”, the system can ask:

What visual test best separates c^* from $c^{(2)}$?

For example:

Confusion pair	Best next discriminative inspection
SMA male vs. SMA female	Center pin / socket crop
SMA vs. RP-SMA	Pin polarity + thread geometry
Straight vs. right-angle	Side-profile / elbow silhouette
Bulkhead vs. non-bulkhead	Mounting nut / flange region
Cable-end vs. board-mount	Rear body / termination geometry

Useful margin definition

Once we have c^* and $c^{(2)}$, define the confidence margin:

$$\Delta_t = \pi_t(c^*) - \pi_t(c^{(2)})$$

Interpretation:

large Δ_t = model is confident
 small Δ_t = model is confused between top two classes

For mobile inference, a practical rule is:

```
if  $\Delta_t \geq \text{margin\_threshold}$ :
    accept  $c^*$ 
else:
    trigger next discriminative inspection against  $c^{(2)}$ 
```

This gives the active inspection loop a very clean decision gate.

ASEM accepts only if all the following hold:

These are the **acceptance gates** for stopping the active inspection loop and committing to the predicted class.

$$\pi_t(c^*) \geq \tau_{\text{accept}}, \pi_t(c^*) - \pi_t(c^{(2)}) \geq \tau_{\text{margin}}$$

Meaning

Condition	Interpretation
$\pi_t(c^*) \geq \tau_{\text{accept}}$	The top class probability must be high enough absolutely.
$\pi_t(c^*) - \pi_t(c^{(2)}) \geq \tau_{\text{margin}}$	The top class must beat the second-best class by a large enough margin.

All rights reserved.

So the detector accepts c^* only when:

```
top probability is high enough
AND
top-vs-runner-up separation is high enough
```

Why both gates matter

Using only top probability can be dangerous.

Example:

Class	Probability
SMA male straight	0.62
RP-SMA male straight	0.58

That example is impossible if probabilities are normalized across all classes, but the idea is still important for multi-head or non-calibrated scores: the top class can look strong while a close competitor is also strong.

For a normalized 10-class softmax, a realistic ambiguous case would be:

Class	Probability
SMA male straight	0.48
RP-SMA male straight	0.43
Other classes	0.09 total

Here:

```
 $\pi_t(c^*) = 0.48$ 
margin = 0.05
```

Even though c^* is the top class, the system should **not accept** yet because the margin is too small.

SMA detector rule

For the connector detector, this becomes:

```
Accept the current connector class only if:
1. the top class probability clears the acceptance threshold, and
2. the top class is sufficiently separated from the second-best class.
```

Otherwise:

```
continue active inspection
```

Practical pseudo-code

```
def should_accept(pi, tau_accept=0.75, tau_margin=0.20):  
    # pi: dict[class_name, probability]  
    ranked = sorted(pi.items(), key=lambda x: x[1], reverse=True)  
  
    c_star, p_star = ranked[0]  
    c_second, p_second = ranked[1]  
  
    accept = (  
        p_star >= tau_accept and  
        (p_star - p_second) >= tau_margin  
    )  
  
    return accept, c_star, c_second, p_star, p_star - p_second
```

Recommended starting thresholds

For an early APK debug build, I would start with:

Threshold	Starting value	Purpose
τ_{accept}	0.70-0.80	Prevent low-confidence acceptance
τ_{margin}	0.15-0.25	Prevent accepting near-ties

A strong initial setting:

```
 $\tau_{\text{accept}} = 0.75$   
 $\tau_{\text{margin}} = 0.20$ 
```

Decision logic

Case	Action
Top probability high, margin high	Accept c^*
Top probability high, margin low	Inspect confusion pair $(c^*, c^{(2)})$
Top probability low, margin high	Inspect broader candidate set; model may be underconfident
Top probability low, margin low	Continue multi-point inspection or request better view

This is the clean stop/continue criterion for the adaptive SMA connector classifier.

and

This is a **required-visual-evidence gate**.

$$\text{ReqVisible}(c^*, D_t) = 1$$

Meaning

The detector may only accept the predicted class c^* if the **required visual evidence** for that class is visible in the current detection/evidence state D_t .

In plain terms:

Do not accept the class just because probability is high.
Accept it only if the required connector-specific feature is actually visible.

Term-by-term

Term	Meaning
c^*	Current top predicted class
D_t	Current evidence state at time t : image crop, detections, keypoints, masks, feature flags, visibility scores
$\text{ReqVisible}(c^*, D_t)$	Boolean function checking whether class-specific required evidence is visible
$= 1$	Required evidence is present / visible / verified

SMA connector interpretation

For SMA/RP-SMA connector detection, different classes require different visual checks.

Predicted class c^*	Required visible evidence
SMA male	Center pin visible
SMA female	Center socket/hole visible
RP-SMA male	Reverse-polarity socket/pin relationship visible
RP-SMA female	Reverse-polarity center pin visible
Right-angle SMA	Elbow/body bend visible
Bulkhead SMA	Nut/flange/threaded mounting region visible
Cable-end SMA	Cable termination/backside body visible
Board-mount SMA	PCB/base mounting geometry visible

So even if:

$$\pi_t(c^*) = 0.91$$

the system should still reject or defer if the required evidence is not visible.

Example:

$c^* = \text{SMA male straight}$
 $\text{ReqVisible}(c^*, D_t) = 0$

Meaning:

The model thinks it is SMA male, but the center pin is not visible enough.
Do not accept yet.

Why this matters

This prevents **overconfident hallucinated classification** from partial views.

For example, if the detector sees only the outer threaded barrel, it may confuse:

SMA male
SMA female
RP-SMA male
RP-SMA female

The outer shell alone is not enough. The model needs the discriminating evidence: **pin/socket polarity**.

Combined acceptance rule

The acceptance stack now becomes:

$$\pi_t(c^*) \geq \tau_{\text{accept}}, \pi_t(c^*) - \pi_t(c^{(2)}) \geq \tau_{\text{margin}}, \text{ReqVisible}(c^*, D_t) = 1$$

Meaning:

Accept c^* only if:

1. confidence is high,
2. margin over second place is wide,
3. required visual evidence is actually visible.

Practical pseudo-code

```
def should_accept(c_star, c_second, pi_t, D_t, tau_accept=0.75,
tau_margin=0.20):
    p_star = pi_t[c_star]
    p_second = pi_t[c_second]

    confidence_ok = p_star >= tau_accept
    margin_ok = (p_star - p_second) >= tau_margin
    required_visible = req_visible(c_star, D_t)

    return confidence_ok and margin_ok and required_visible
```

This is the right kind of guardrail for a mobile connector detector: it forces the model to justify high confidence with actual class-specific visual evidence.

Meaning the evidence needed for the claim is actually visible.

This gives the active detector a **sequential decision-theoretic stopping rule**, similar in spirit to a **Sequential Probability Ratio Test**, but adapted to the current top-two posterior classes.

$$\Lambda_t = \log \frac{\pi_t(c^*)}{\pi_t(c^{(2)})}$$

Meaning

Λ_t is the **log-odds advantage** of the current best class over the second-best class.

Value	Meaning
$\Lambda_t > 0$	Top class is more likely than second class
$\Lambda_t = 0$	Top two classes are tied
Large Λ_t	Strong evidence for c^* over $c^{(2)}$
Small Λ_t	Ambiguous top-two race

Because this uses a ratio, it behaves more sharply than a raw probability margin.

For example:

$\pi_t(c^*)$	$\pi_t(c^{(2)})$	Odds	Λ_t
0.60	0.30	2:1	0.693
0.80	0.10	8:1	2.079
0.95	0.02	47.5:1	3.861
0.995	0.005	199:1	5.293

SPRT-like decision rule

Accept if $\Lambda_t \geq \log A$, Continue if $\log B < \Lambda_t < \log A$

Operationally:

```

if  $\Lambda_t \geq \log(A)$ :
    accept  $c^*$ 
elif budget_exhausted:
    abstain / request better view / return uncertain
else:
    continue active inspection

```

Where:

Threshold	Meaning
A	Acceptance odds threshold
B	Lower continuation/weak-evidence threshold
$\log A$	Log-odds level needed to accept
$\log B$	Lower bound below which evidence is too weak or unstable

Practical odds thresholds

The proposed tuning ranges are sensible:

Parameter	Range	Interpretation
τ_{accept}	0.995–0.999	Very strict for full connector type + gender/polarity
τ_{margin}	0.20–0.35	Requires meaningful separation from runner-up
A	odds 20–100	Requires top class to be 20× to 100× stronger than second
B	odds 1/3–1	Controls how aggressively the detector keeps probing

Converted to logs:

Odds	Log threshold
A = 20	$\log A \approx 2.996$
A = 50	$\log A \approx 3.912$
A = 100	$\log A \approx 4.605$
B = 1	$\log B = 0$
B = 1/3	$\log B \approx -1.099$

Important implementation detail

Use a small epsilon for numerical stability:

```
Lambda_t = math.log((pi[c_star] + eps) / (pi[c_second] + eps))
```

Otherwise, a near-zero second-class posterior can create unstable or infinite odds.

Combined final acceptance stack

The detector should not accept solely from the SPRT-like odds rule. I would combine all gates:

$$\pi_t(c^*) \geq \tau_{\text{accept}}, \pi_t(c^*) - \pi_t(c^{(2)}) \geq \tau_{\text{margin}}, \Lambda_t \geq \log A, \text{ReqVisible}(c^*, D_t) = 1$$

So the final decision is:

Accept c^* only if:

1. top posterior is high enough,
2. top-vs-second margin is high enough,
3. top-vs-second log odds clear the acceptance boundary,
4. required visual evidence for c^* is visible.

Suggested mobile-debug starting values

For a first real-phone validation pass, I would start with:

```

 $\tau_{\text{accept}}$  = 0.995
 $\tau_{\text{margin}}$  = 0.25
A = 50
B = 1/3
max_inspections = 5 to 7
eps = 1e-8

```

All rights reserved.

That gives:

$\log A \approx 3.912$
 $\log B \approx -1.099$

Practical policy

```

def active_decision(
    pi_t,
    c_star,
    c_second,
    D_t,
    budget_used,
    max_budget,
    tau_accept=0.995,
    tau_margin=0.25,
    A=50.0,
    B=1/3,
    eps=1e-8,
):
    p_star = pi_t[c_star]
    p_second = pi_t[c_second]

    margin = p_star - p_second
    lambda_t = math.log((p_star + eps) / (p_second + eps))

    confidence_ok = p_star >= tau_accept
    margin_ok = margin >= tau_margin
    odds_ok = lambda_t >= math.log(A)
    visible_ok = req_visible(c_star, D_t)

    if confidence_ok and margin_ok and odds_ok and visible_ok:
        return {
            "decision": "accept",
            "class": c_star,
            "lambda_t": lambda_t,
            "margin": margin,
            "p_star": p_star,
            "p_second": p_second,
        }

    if budget_used >= max_budget:
        return {
            "decision": "abstain",
            "reason": "budget_exhausted_before_acceptance",
            "candidate": c_star,
            "runner_up": c_second,
            "lambda_t": lambda_t,
            "margin": margin,
            "p_star": p_star,
            "p_second": p_second,
        }

    if math.log(B) < lambda_t < math.log(A):
        return {

```

```

        "decision": "continue",
        "reason": "insufficient_odds_separation",
        "candidate": c_star,
        "runner_up": c_second,
        "lambda_t": lambda_t,
        "margin": margin,
    }

    return {
        "decision": "continue",
        "reason": "weak_or_unstable_top_class",
        "candidate": c_star,
        "runner_up": c_second,
        "lambda_t": lambda_t,
        "margin": margin,
    }

```

Why this is better than confidence alone

The strong part of this formulation is that it separates three things that are often incorrectly collapsed into one softmax score:

Gate	Protects against
τ_{accept}	Low absolute confidence
τ_{margin}	Near-tie ambiguity
$\Lambda_t \geq \log A$	Weak odds advantage over runner-up
ReqVisible	High-confidence hallucination from missing visual evidence
Budget exhaustion	Infinite probing / mobile latency blowups

For the SMA detector, this is exactly the right shape: **high confidence is necessary but not sufficient**. The model must also show top-two separation, odds dominance, and visible class-specific evidence.

Abstention mapping

If no acceptable probe remains, or if the remaining probes all have expected gain below ϵ_{probe} , then the system abstains and maps the reason:

- return unsupported_connector if $\pi_t(\perp)$ exceeds its threshold;
- return need_second_angle if required evidence is pose-limited;
- return need_scale_reference if surviving hypotheses differ mainly by scale-sensitive geometry;
- return ambiguous otherwise, with top alternatives.

Formally, if

$$\max_{j \in \mathcal{J}_{\text{avail}}} \frac{\mathbb{E}[H(\pi_t) - H(\pi_{t+1}|z_j)]}{\text{cost}(j)} < \epsilon_{\text{probe}},$$

then stop probing and abstain according to the dominant unmet-evidence reason.

Pseudocode

```

def asem_infer(x):
    det = detector(x)
    roi = crop(x, det.bbox)

    q_support = support_gate(roi)          # p(in_support), p(unsupported)
    p_flat = calibrate(flat_classifier(roi))

    pi = initialize_posterior(p_flat, q_support)
    K = candidate_closure(top_k(pi, k=3), include_unsupported=True)

    if accept(pi, K, evidence=None):
        return nailed_it(argmax(pi), pi)

    attr = attribute_refiner(roi)
    pi = fuse_attributes(pi, attr, K)

    used = []
    for _ in range(MAX_AUTOMATIC_PROBES):
        if accept(pi, K, evidence=used):
            return nailed_it(argmax(pi), pi)

        j = choose_probe_by_eig_over_cost(pi, K, roi, used)
        if j is None:
            break

        z = run_probe(j, roi)
        pi = posterior_update(pi, z, j, K)
        used.append((j, z))

    if unsupported_ready(pi):
        return unsupported(pi, used)
    if angle_needed(pi, used):
        return need_second_angle(pi, used)
    if scale_needed(pi, used):
        return need_scale_reference(pi, used)
    return ambiguous(top_alternatives(pi), used)

```

Numerical walkthrough

Consider a supported but hard case where the flat classifier is confused and initially favors the wrong gender:

Initial calibrated posterior after flat classifier and support gate:

- 2.4mm-M: 0.54
- 2.4mm-F: 0.18
- 2.92mm-M: 0.12

All rights reserved.

- 2.92mm-F: 0.08
- SMA-F: 0.04
- $\perp$: 0.02
- others: 0.02

Candidate closure retains {2.4mm-M, 2.4mm-F, 2.92mm-M, 2.92mm-F, SMA-F, $\perp$ }.

The center probe sees a **socket** with high visibility. Suppose the resulting likelihood ratios are approximately:

- female classes: $\times 5$
- male classes: $\times 0.15$
- unsupported: $\times 1.2$

After normalization, the posterior becomes approximately:

- 2.4mm-F: 0.57
- 2.92mm-F: 0.21
- SMA-F: 0.10
- 2.4mm-M: 0.07
- 2.92mm-M: 0.03
- $\perp$: 0.02

Now the dominant confusion is 2.4mm-F vs 2.92mm-F, so the ring probe has the highest expected information gain. Suppose the ring probe yields a normalized annulus-aperture ratio more compatible with 2.4 mm than 2.92 mm, with likelihood ratios:

- 2.4mm-F: $\times 3.0$
- 2.92mm-F: $\times 0.8$
- SMA-F: $\times 0.5$
- others: near neutral or worse

After normalization:

- 2.4mm-F: 0.83
- 2.92mm-F: 0.11
- SMA-F: 0.03
- others: 0.03 total

If $\tau_{\text{accept_full}} = 0.995$, this is still not enough to say “nailed it.” The body-span or axial probe runs next. Suppose the body-span evidence supports 2.4 mm over 2.92 mm by another factor of 2.5. Then the posterior may reach:

- 2.4mm-F: 0.96
- 2.92mm-F: 0.03

- others: 0.01

At this point a family-only claim is likely safe, but a full “nailed it” still depends on the chosen thresholds. If the product target is extremely conservative, the system might still ask for a second angle or scale reference. If the validation-calibrated $\tau_{\text{accept_full}}$ is lower, it may accept. The key point is that the app is no longer forced to stop at the first weak top-1 guess.

For the hotel-TV screenshot I supplied, ASEM would likely behave differently. Because the current support set contains only the ten precision/SMA classes, a support gate plus later structural probes would probably drive probability mass toward $\perp$ or toward an insufficiency outcome instead of allowing a moderate-confidence 2.4mm-M answer to escape as the final app result.

Relation to finite element analysis

The FEA analogy is useful as an intuition for **adaptive refinement**, but it should remain an analogy only.

In FEA or finite-difference analysis, the mesh discretizes a physical domain to solve field equations. ASEM is not solving PDEs or building a numerical simulation mesh over the connector image. Instead, it incrementally allocates **semantic evidence points**—localized visual tests and landmarks—where uncertainty remains highest. The common idea is refinement where needed. The mathematical object is different.

In ASEM, the “mesh” is an adaptive set of evidence probes selected to reduce classification risk subject to compute and usability constraints, *not a discretization of a continuum physics problem.*

This is a bit much to digest, however, this is how a spec should be written in proposal format formally so when developing the codebase and doing major modifications to the repo, it can be done in a modularized and very structured manner, and often times while diving in we determine that we may have missed something, or added in too much and needs to be shaved, or total omissions and additions.

One thing is clear, to offer the customer a fully functioning highly accurate mobile app, the above is most likely going to be the best pathway forward after some preliminary testing.

Please feel free to message or email me any questions, proposed changes, additions, concerns, anything and I will review thoroughly and respond.

Moving into rev 3 and will write a report on results asap.

Dr. T. Jerry Mahabub, Ph.D.

References

[1] hotdogornot/training/docs/classifier_journey.md at master · trextrader/hotdogornot · GitHub

https://github.com/trextrader/hotdogornot/blob/master/training/docs/classifier_journey.md?utm_source=chatgpt.com

[2] [6] [9] [14] [18] [19] [21] hotdogornot/IMPLEMENTATION_PLAN.md at master · trextrader/hotdogornot · GitHub

https://github.com/trextrader/hotdogornot/blob/master/IMPLEMENTATION_PLAN.md?utm_source=chatgpt.com

[3] [16] hotdogornot/exports/mobile/README.md at master · trextrader/hotdogornot · GitHub

https://github.com/trextrader/hotdogornot/blob/master/exports/mobile/README.md?utm_source=chatgpt.com

[4] [5] hotdogornot/docs/CONNECTOR_TAXONOMY.md at master · trextrader/hotdogornot · GitHub

https://github.com/trextrader/hotdogornot/blob/master/docs/CONNECTOR_TAXONOMY.md?utm_source=chatgpt.com

[7] [1706.04599] On Calibration of Modern Neural Networks

https://arxiv.org/abs/1706.04599?utm_source=chatgpt.com

[8] Learning to Navigate for Fine-grained Classification

https://arxiv.org/abs/1809.00287?utm_source=chatgpt.com

[10] raw.githubusercontent.com

https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/rfconnectorai/classifier/model_multihead.py?utm_source=chatgpt.com

[11] raw.githubusercontent.com

https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/rfconnectorai/schemas/instance.py?utm_source=chatgpt.com

[12] raw.githubusercontent.com

https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/rfconnectorai/specs/connectors.yaml?utm_source=chatgpt.com

[13] [1607.03691] Sequential Cost-Sensitive Feature Acquisition

https://arxiv.org/abs/1607.03691?utm_source=chatgpt.com

[15] [raw.githubusercontent.com](https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/configs/classes.yaml)

<https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/configs/classes.yaml>?utm_source=chatgpt.com

[17] [raw.githubusercontent.com](https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/rfconnectorai/data/build_yolo_dataset.py)

https://raw.githubusercontent.com/trextrader/hotdogornot/master/training/rfconnectorai/data/build_yolo_dataset.py?utm_source=chatgpt.com

[20] [hotdogornot/docs/MODEL_TRAINING_PIPELINE_SPEC.md](https://github.com/trextrader/hotdogornot/blob/master/docs/MODEL_TRAINING_PIPELINE_SPEC.md) at master · trextrader/hotdogornot · GitHub

https://github.com/trextrader/hotdogornot/blob/master/docs/MODEL_TRAINING_PIPELINE_SPEC.md?utm_source=chatgpt.com